Week 6 - Analytics, Databases & Backends

1. Introduction

To implement a simple automated method to test a tool, store the test result as data that is then queried and translated into a graph using matplotlib, I added to my Jenkins pipeline and had it run a Python script. My Jenkins job already automatically builds the Unreal Engine project periodically to make sure the staging branch is in working order. I chose Jenkins as the tool to work on for this task as it already outputs the majority of it's data to Discord so I know that it is definitely possible to translate those results onto a graph.

Before implementing the automated output graph to the Jenkins pipeline, I first tested completing this task on some data found online on Kaggle (Kaggle: The World’s AI Proving Ground, s.d.) a website that stores and displays data on endless topics. This test was used to learn how to generate structured datat outputs and graphs using Python and matplotlib. The dataset from Kaggle contained information about comic books such as the title, publisher, release year, format and genre.

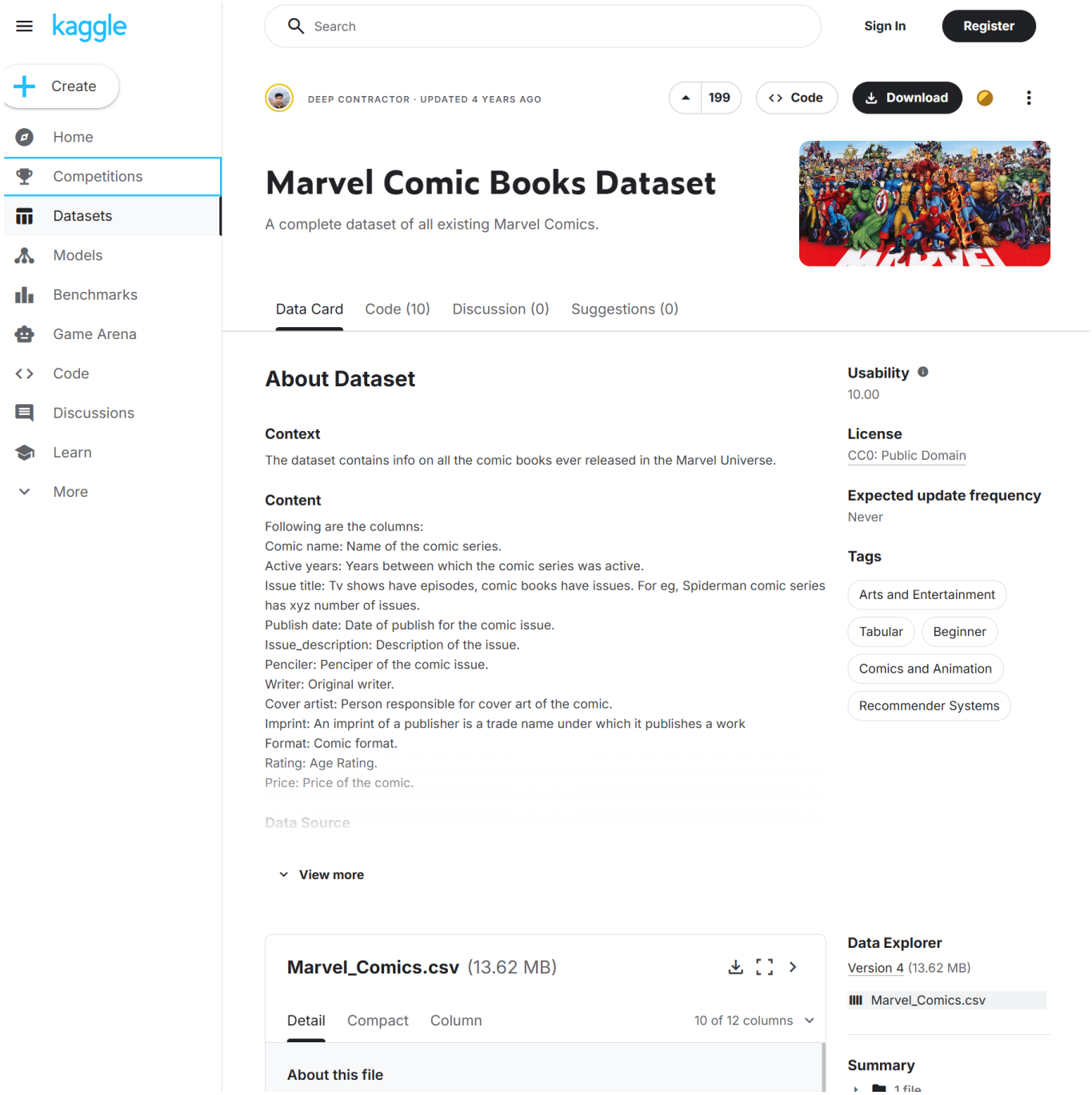

Figure 1. The Kaggle website that provided the data that I used to practice this task on.

---

2. Implementation

The comic book dataset test was done first to test the process of converting data into a structured format and generating graphs from the data. Firstly, I found myself a data set that I wanted to test on which ended up being data on Marvel comics. Then I used AI to create me a prompt that I would give to the antigravity (Google Antigravity, s.d.) claude opus AI which would then generate Python scripts capable of processing the dataset and producing analysis outputs. The script generated from the AI cleaned the dataset by removing any duplicate data, standardising the publishers, genres and format fields and converting release year values to numeric data. This resulted in a clear dataset containing 9871 entries across 25 publishers and 15 genres. Several graphs were then generated using matplotlib. These graphs included releases per year, publisher distribution, genre distribution and format trends. These visualisations demonstrated how the dataset could be analysed and interpreted programmatically.

Comic Books Dataset — Analysis Report

1. Dataset Overview

Metric	Value
Total records (after cleaning)	9,870
Year range	1938–2025
Unique publishers	25
Unique genres	15
Unique formats	6

This dataset contains information on comic books including their titles, publishers, release years, formats, and genres.

2. Data Cleaning Summary

The following cleaning steps were applied to the raw dataset:

- Detected 463 missing values across columns:
- ○ Title: 31 missing
- ○ Publisher: 42 missing
- ○ ReleaseYear: 313 missing
- ○ Format: 28 missing
- ○ Genre: 49 missing
- Removed 99 duplicate rows.
- Standardised 'Publisher': reduced unique values from 105 to 25 (strip + title case).
- Standardised 'Format': reduced unique values from 18 to 6 (strip + title case).
- Standardised 'Genre': reduced unique values from 60 to 15 (strip + title case).
- Corrected acronym-based publisher names (DC, IDW, AWA, MLT).
- Stripped whitespace from 'Title'.
- Converted ReleaseYear to numeric; 51 non-numeric values coerced to NaN.
- Dropped 31 rows with missing Title.
-

Cleaning complete. Rows: 10,000 → 9,870 (130 removed).

3. Key Findings

- The dataset spans from **1938–2025**, with the peak release year being **2020** (258 releases).
- **Marvel Comics** is the dominant publisher with **2,605** entries, followed by **DC Comics** with **2,375** entries.
- **Single Issue** is the most common format, accounting for 5,398 entries (54.7% of the dataset).
- **Superhero** is the dominant genre with 3,120 entries.
- The comic book industry shows strong growth from the 1990s onward, with the 2000s and 2010s representing the bulk of releases.

Figure 2. A few of the data that was generated based on the Marvel data from Kaggle.

After the workflow for this task was completed on data stored online, I used a similar approach on my Jenkins pipeline. The Jenkins pipeline was adjusted so that after each automated build, it writes the build number, timestamp, branch name, build result and build duration to a CSV file. A python script that I created then reads this file using pandas and calculates the statistics such as total builds, success rates and average build duration. The script then generates a matplotlib graph showing build duration over time. This approach automates the process of collecting and analysing build performance data.

```
import os
import pandas as pd
import matplotlib.pyplot as plt
```

```python
results_dir = r"#############"

csv_file = os.path.join(results_dir, "build_results.csv")
duration_graph = os.path.join(results_dir, "build_duration.png")
summary_file = os.path.join(results_dir, "build_summary.txt")

if not os.path.exists(csv_file):
    print("CSV file not found")
    raise SystemExit(1)

df = pd.read_csv(csv_file)

df["BuildNumber"] = pd.to_numeric(df["BuildNumber"], errors="coerce")
df["DurationSeconds"] = pd.to_numeric(df["DurationSeconds"], errors="coerce")
df = df.dropna(subset=["BuildNumber", "DurationSeconds"])

if df.empty:
    print("CSV is empty after cleaning")
    raise SystemExit(1)

total_builds = len(df)
success_count = (df["Result"] == "SUCCESS").sum()
failure_count = (df["Result"] == "FAILURE").sum()
avg_duration = df["DurationSeconds"].mean()
success_rate = (success_count / total_builds) * 100 if total_builds > 0 else 0

with open(summary_file, "w", encoding="utf-8") as f:
    f.write(f"Total builds: {total_builds}\n")
    f.write(f"Successful builds: {success_count}\n")
    f.write(f"Failed builds: {failure_count}\n")
    f.write(f"Average duration: {avg_duration:.2f} seconds\n")
    f.write(f"Success rate: {success_rate:.2f}%\n")

plt.figure(figsize=(8, 5))
plt.plot(df["BuildNumber"], df["DurationSeconds"], marker="o")
plt.xlabel("Build Number")
plt.ylabel("Duration in Seconds")
plt.title("Jenkins Build Duration Over Time")
plt.grid(True)
plt.tight_layout()
plt.savefig(duration_graph)
plt.close()

print("Saved graph to:", duration_graph)
print("Saved summary to:", summary_file)
```

Figure 3. The Python script that analyses the build results before producing a CSV file, build duration graph and summary text file

---

3. Outcome

The final system automatically tests the Jenkins build pipeline and records the results of each run as structured data. Each time Jenkins performs a build, the pipeline logs the outcome to a CSV file and runs the Python analysis script which then produces a graph showing build duration over time and generates a summary file containing the calculated statistics such as success rate and average build duration.

```
BuildNumber,DateTime,Branch,Result,DurationSeconds
40,16/03/2026 12:58,staging,SUCCESS,50
41,16/03/2026 13:01,staging,SUCCESS,23
42,16/03/2026 13:03,staging,SUCCESS,23
43,16/03/2026 13:09,staging,SUCCESS,163
44,16/03/2026 13:18,staging,SUCCESS,151
45,16/03/2026 13:30,staging,SUCCESS,152
46,2026-03-16 14:17:10,staging,SUCCESS,184
47,2026-03-16 14:24:31,staging,SUCCESS,236
49,2026-03-16 14:30:45,staging,SUCCESS,160
50,2026-03-17 22:17:12,staging,FAILURE,1024
51,2026-03-17 22:19:46,staging,ABORTED,28
52,2026-03-17 22:22:51,staging,SUCCESS,172
53,2026-03-18 22:03:31,staging,SUCCESS,200
54,2026-03-19 22:03:27,staging,SUCCESS,198
55,2026-03-22 22:59:38,staging,SUCCESS,171
56,2026-03-24 22:32:23,staging,SUCCESS,204
57,2026-03-26 18:34:24,staging,FAILURE,90
58,2026-03-26 18:50:41,staging,SUCCESS,292
59,2026-03-28 23:42:51,staging,SUCCESS,206
60,2026-03-31 22:18:31,staging,SUCCESS,214
```

	A	B	C	D	E	F	G
1	BuildNumb	DateTime	Branch	Result	DurationSeconds		
2	40	16/03/2026 12:58	staging	SUCCESS	50		
3	41	16/03/2026 13:01	staging	SUCCESS	23		
4	42	16/03/2026 13:03	staging	SUCCESS	23		
5	43	16/03/2026 13:09	staging	SUCCESS	163		
6	44	16/03/2026 13:18	staging	SUCCESS	151		
7	45	16/03/2026 13:30	staging	SUCCESS	152		
8	46	16/03/2026 14:17	staging	SUCCESS	184		
9	47	16/03/2026 14:24	staging	SUCCESS	236		
10	49	16/03/2026 14:30	staging	SUCCESS	160		
11	50	17/03/2026 22:17	staging	FAILURE	1024		
12	51	17/03/2026 22:19	staging	ABORTED	28		
13	52	17/03/2026 22:22	staging	SUCCESS	172		
14	53	18/03/2026 22:03	staging	SUCCESS	200		
15	54	19/03/2026 22:03	staging	SUCCESS	198		
16	55	22/03/2026 22:59	staging	SUCCESS	171		
17	56	24/03/2026 22:32	staging	SUCCESS	204		
18	57	26/03/2026 18:34	staging	FAILURE	90		
19	58	26/03/2026 18:50	staging	SUCCESS	292		
20	59	28/03/2026 23:42	staging	SUCCESS	206		
21	60	31/03/2026 22:18	staging	SUCCESS	214		
22							
23							
24							
25							

Figure 4 & 5. The CSV table outputted by Jenkins after the most recent build.

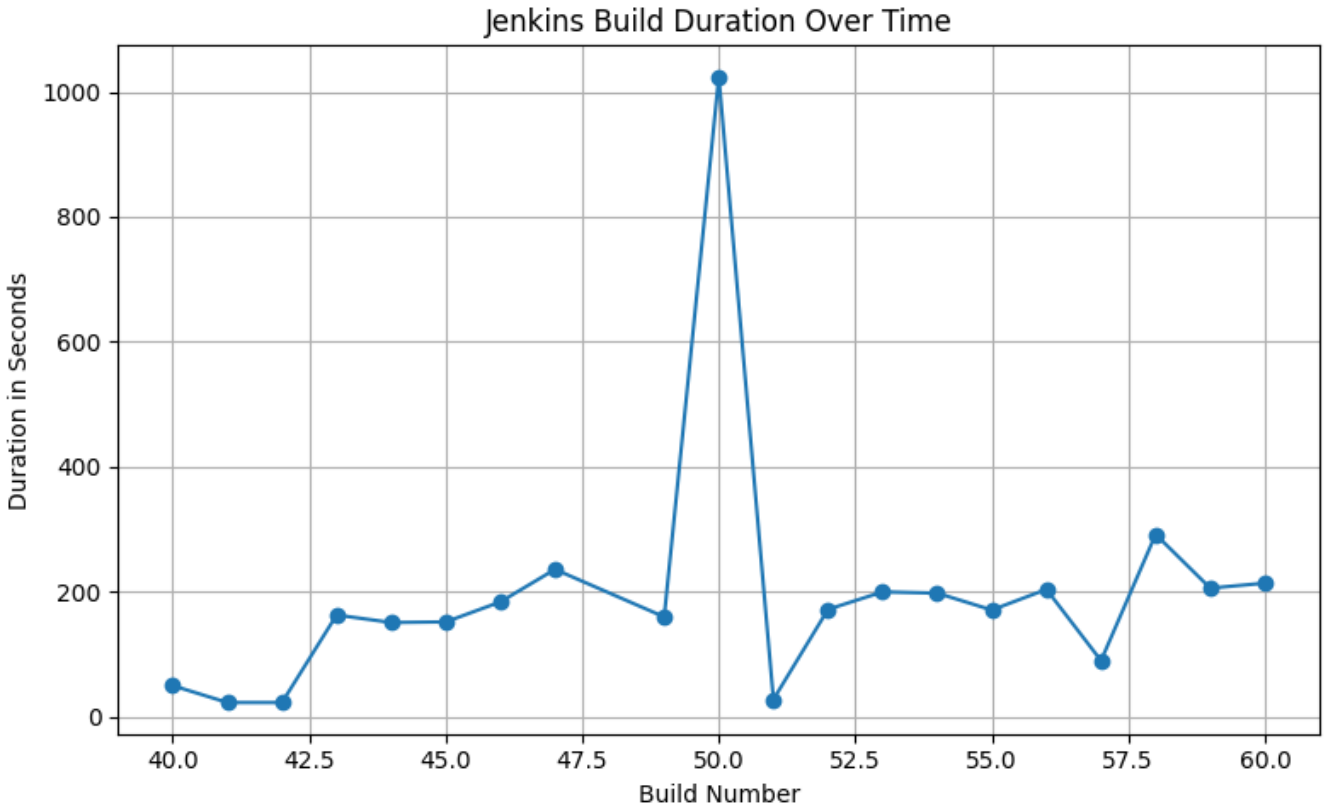

Figure 6. The build duration graph outputted by Jenkins after the most recent build

```
Total builds: 20
Successful builds: 17
Failed builds: 2
Average duration: 197.05 seconds
Success rate: 85.00%
```

Figure 7. The results outputted after the most recent Jenkins build. Since a build was aborted, the successful builds and failed builds don't add up to the total builds.

---

4. Bibliography

- Kaggle: The World's AI Proving Ground (s.d.) At: https://www.kaggle.com/ (Accessed 16/03/2026).
- Google Antigravity (s.d.) At: https://antigravity.google/ (Accessed 16/03/2026).

AI Usage Declaration

- Chatgpt was used for paragraph structure and spelling and grammar checking and the process of creating the API.
- ChatGPT (s.d.) At: https://chatgpt.com/ (Accessed 09/02/2026).